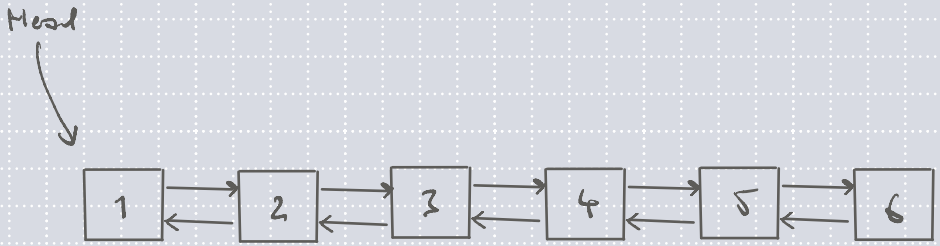
Today

- **Deadlock and Concurrency**
- **R3000 and Assembly**
- **Threads**
- **Kernel Entry and Exit**
- **Week 3 Tutorial**
  Questions 7*, 8-11
- **Week 4 Tutorial**
  Questions 1-6, 9-12, 14, 15

7. Describe (and give pseudocode for) a synchronised linked list structure based on thread list code in the OS/161 codebase (kern/thread/threadlist.c).



(Skipped. See tutorial answers when they are posted.)

The thread subsystem in OS/161 uses a linked list of threads to manage some of its state (kern/thread/threadlist.c). This structure is not synchronised. Why not?

Concurrency and Deadlock

8. For each of the following scenarios, one or more dining philosophers are going hungry. What is the condition the philosphers are suffering from?

(a) Each philosopher at the table has picked up his left fork, and is waiting for his right fork?

Deadlock (no activity, no progress)

(b) Only one philosopher is allowed to eat at a time. When more than one philospher is hungry, the youngest one goes first. The oldest philosopher never gets to eat.

Starvation (progress made, but not for all)

(c) Each philosopher, after picking up his left fork, puts it back down if he can't immediately pick up the right fork to give others a chance to eat. No philosopher is managing to eat despite lots of left fork activity.

Live lock (constant activity, but no progress)

9. What is starvation? Give an example.

Progress is made, but some work is never actioned.

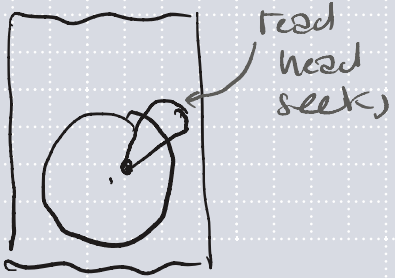
(Different to tutorial example.)

Action 'small jobs' before 'big jobs'; if we have an endless number of 'small jobs', we may never action 'big jobs'.

10. Two processes are attempting to read independent blocks from a disk, which involves issuing a *seek* command and a *read* command. Each process is interrupted by the other between its *seek* and *read*. When a process discovers the other process has moved the disk head, it re-issues the original seek to re-position the head for itself, which is again interrupted prior to read. This alternate seeking continues indefinitely, with neither process able to read their data from the disk. Is this deadlock, starvation, or livelock? How would you change the system to prevent the

Livelock!

Use a lock to perform both seek and then read.



11. Describe four ways to *prevent* deadlock by attacking the conditions required for deadlock.

- Mutual exclusion (make resource sharable) ← often not possible
- Hold and wait (all resources acquired initially) ← prone to livelock
- No preemption (take resource forcibly) ← something's can't be interrupted
- Circular wait (enforce global lock ordering) ← Normally the best choice

12. Deadlock avoidance.

See Section 6.5 of Modern Operating Systems (Tanenbaum, Bos) and Week 3 Tutorial Solutions. Digital version is available from UNSW Library.

13. Alternative solution for Dining Philosophers problem.

Try this one yourself!

1. What is *branch delay*? (see p.19 of R3000 manual)

Execution of a jump is delayed until after the next instruction.

2. (a) **arg1** returns its return value in what register?

v0

2. (c) What does **jr ra** do?

Jumps to the return address of the function.

2. (e) Why is the **move** instruction in **arg1** after the **jr** instruction?

To perform work in the branch delay slot.

```
int arg1(int a)
{
  return a;
}

004000f0 <arg1>:
  4000f0:       03e00008        jr      ra
  4000f4:       00801021        move    v0,a0
```

2. (b) Why are there no stack references in **arg2**?

No local variables – no stack necessary.

2. (d) Which register contains the first argument to the function?

a0

```
int arg2(int a, int b)
{
  return a + b;
}

004000f8 <arg2>:
  4000f8:       03e00008        jr      ra
  4000fc:       00851021        addu    v0,a0,a1


int arg5(int a, int b, int c, int d, int e )
{
  return a + b + c + d + e;
}

int arg6(int a, int b, int c, int d, int e, int f)
{
  return a + b + c + d + e + f;
}
```

2. (f) Why does **arg5** and **arg6** reference the stack?

The 5th and 6th arguments are passed on the stack.

```
0040011c <arg5>:
  40011c:       00852021        addu    a0,a0,a1
  400120:       00863021        addu    a2,a0,a2
  400124:       00c73821        addu    a3,a2,a3
  400128:       8fa20010        lw      v0,16(sp)
  40012c:       03e00008        jr      ra
  400130:       00e21021        addu    v0,a3,v0

00400134 <arg6>:
  400134:       00852021        addu    a0,a0,a1
  400138:       00863021        addu    a2,a0,a2
  40013c:       00c73821        addu    a3,a2,a3
  400140:       8fa20010        lw      v0,16(sp)
  400144:       00000000        nop
  400148:       00e22021        addu    a0,a3,v0
  40014c:       8fa20014        lw      v0,20(sp)
  400150:       03e00008        jr      ra
  400154:       00821021        addu    v0,a0,v0
```

e

loaded variable not available until following instruction

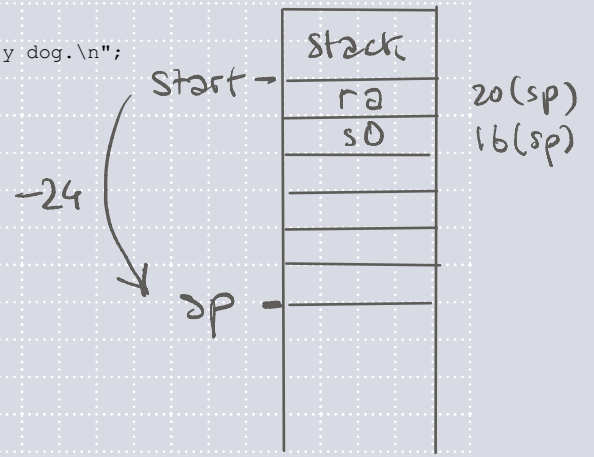
f

```c
#include <stdio.h>
#include <unistd.h>

char teststr[] = "\nThe quick brown fox jumps of the lazy dog.\n";

void reverse_print(char *s)
{
    if (*s != '\0') {
        reverse_print(s + 1);
        write(STDOUT_FILENO, s, 1);
    }
}

int main(void)
{
    reverse_print(teststr);
}
```



3. (a) Describe what each line of code is doing.

```
004000f0 <reverse_print>:
  4000f0:       27bdffe8        addiu   sp,sp,-24
  4000f4:       afbf0014        sw      ra,20(sp)
  4000f8:       afb00010        sw      s0,16(sp)
  4000fc:       80820000        lb      v0,0(a0)
  400100:       00000000        nop
  400104:       10400007        beqz    v0,400124 <reverse_print+0x34>
  400108:       00808021        move    s0,a0
  40010c:       0c10003c        jal     4000f0 <reverse_print>
  400110:       24840001        addiu   a0,a0,1
  400114:       24040001        li      a0,1
  400118:       02002821        move    a1,s0
  40011c:       0c1000af        jal     4002bc <write>
  400120:       24060001        li      a2,1
  400124:       8fbf0014        lw      ra,20(sp)
  400128:       8fb00010        lw      s0,16(sp)
  40012c:       03e00008        jr      ra
```

} prologue! alloc 24 bytes

} recurse

} write syscall

} epilogue! free 24 bytes

load delay

3. (b) What is the maximum depth the stack can grow to when this function is called?

Each recursive call allocates 24 bytes, so length of s times 24.

4. Why is recursion or large arrays of local variables avoided by kernel programmers?

Kernel stacks are finite.

Threads

5. Compare cooperative versus premptive multithreading.

Cooperative: explicit yields required ← (threads must cooperate)

Premptive: no thread may dominate

6. Describe *user-level threads* and *kernel-level threads*. What are the advantages or disadvantages?

user-level: + performant (no context switch)
- not parallelisable on multicore

kernel-level: + parallelisable on multicore

Kernel Entry and Exit

(see R3000 reference manual)

9. What is the EPC register? What is it used for?

p.41

Contains the program counter before the exception occured. Allows the program to continue where it left off after an exception.

10. What happens to the KUc and IEc bits in the STATUS register when an exception occurs? Why? How are they stored?

p.39

layout on p.37

$KU_p = KU_c$ $IE_p = IE_c$

Stored in status register; swapped back after exception.

11. What is the value of the ExcCode in the Cause register

p.40

8

12. Why must kernel programmers be especially careful when implementing system calls?

If arguments aren't validated, things can go wrong.

E.g. user passes pointer which points to kernel-only memory

14. In the example given in lectures, the library function read invoked the read system call. Is it essential that both have the same name? If not, which name is important?

userspace signature matters; kernel does not

(syscall no. identifies syscall in kernel)

15. To a programmer, a system call looks like any other call to a library function. Is it important that a programmer know which library functions result in system calls? Under what

Context switching is expensive; if performance matters, it's preferable to avoid syscalls if possible.